

OPTIMASI GERAKAN ROBOT SEPAK BOLA MENGUNAKAN *KALMAN* *FILTER* PADA *TRACKING* OBJEK BERBASIS YOLO

Fikri Rivandi | 2207112583



Universitas Riau
Program Studi S1 Teknik Informatika

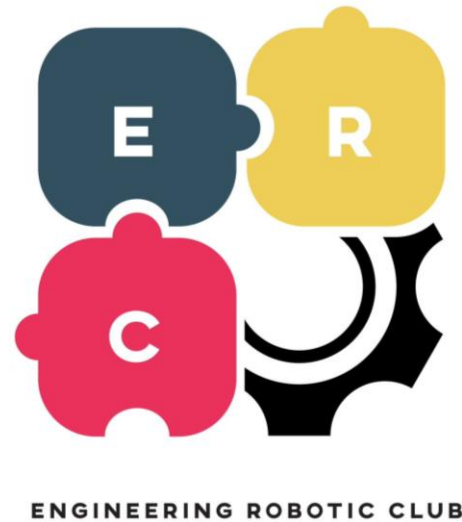
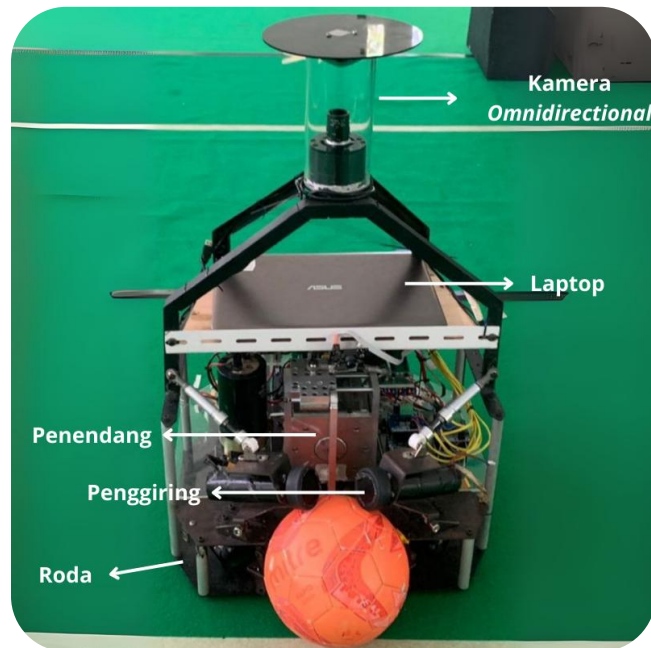


BAB I

Pendahuluan

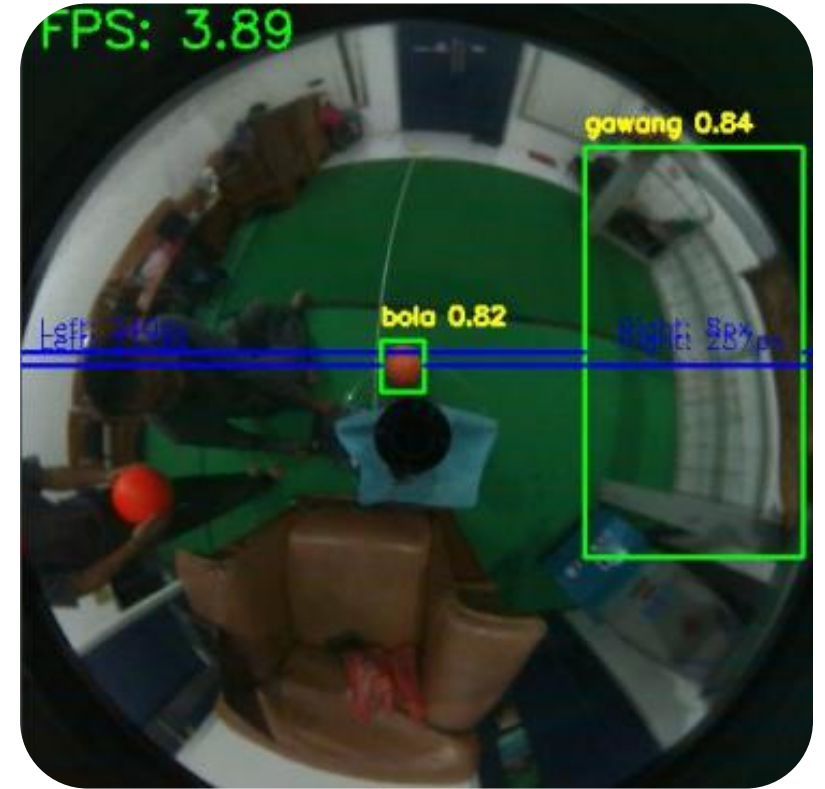
Latar Belakang (1)

- Kontes Robot Sepak Bola Beroda Indonesia (KRSBI) menuntut robot yang cepat, akurat, dan mandiri
- Tracking bola menjadi komponen paling krusial
- Tim ERC Universitas Riau telah berprestasi (tingkat nasional 2021-2022)
- Namun, regulasi baru & tantangan teknis terus muncul



Latar Belakang (2) – Permasalahan

- *Hardware* terbatas (Laptop ASUS K401U: i5-7200U, 8GB RAM, GPU 940MX)
- YOLOv8 berjalan setiap *frame* → FPS rendah (13.6 FPS)
- Gerakan robot *jitter* (goyang tidak stabil)
- Sering terjadi oklusi (bola tertutup) → *tracking* hilang
- Akibat: Robot lambat merespons, sering gagal mengejar bola



Rumusan Masalah

- Bagaimana meningkatkan kestabilan tracking dan FPS pada *hardware* terbatas?
- Bagaimana mengurangi dampak oklusi agar *tracking* bola tetap kontinu?

Tujuan Penelitian

- Mengembangkan sistem *tracking* yang lebih stabil dan responsif menggunakan *Kalman Filter*
- Meningkatkan ketahanan *tracking* terhadap oklusi sehingga gerakan robot lebih efisien dan akurat

Batasan Penelitian

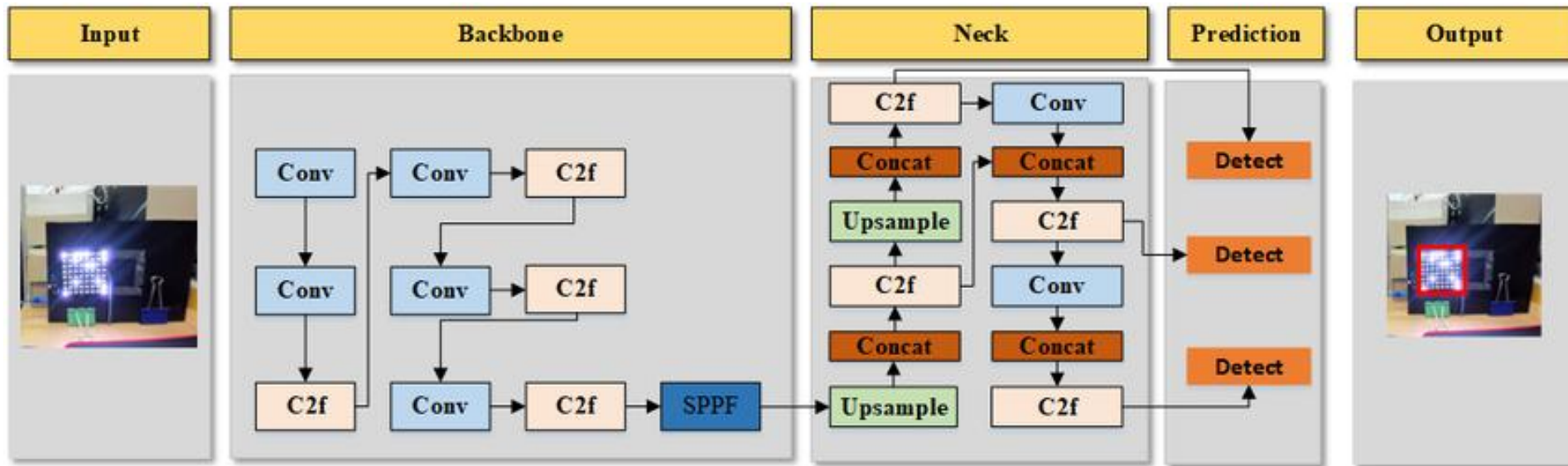
- Hanya fokus pada *tracking* bola menggunakan kamera *omnidirectional*
- Menggunakan YOLOv8 (tanpa modifikasi arsitektur)
- Estimasi posisi menggunakan *Kalman Filter* saja
- Tidak membahas strategi tim atau komunikasi antar robot

BAB II

Tinjauan Pustaka

Landasan Teori – YOLOv8

- YOLOv8 adalah model *object detection* yang akurat (Redmon et al., 2016)
- Kelemahan: Komputasi berat untuk *hardware low-end* (Miharja et al., 2025)
- Solusi: Tidak dijalankan setiap *frame*



Landasan Teori – *Kalman Filter*

- Algoritma estimasi rekursif untuk sistem dinamis
- Memprediksi posisi objek di frame berikutnya
- Sangat ringan komputasinya
- Cocok untuk mengatasi oklusi

Solusi yang Diusulkan

Arsitektur *Optimized*:

- YOLOv8 → Deteksi bola (hanya tiap 3 *frame*)
 - *Frame Skipping* (SL = 3)
 - *Kalman Filter* → Melakukan prediksi saat YOLO tidak aktif atau saat oklusi
- => **Konsep:** *Tracking-by-Detection + Prediction*

Perbandingan Sistem

- Perbandingan sistem yang digunakan dapat dilihat pada tabel berikut:

Aspek	<i>Baseline</i>	<i>Optimized</i>
YOLOv8	Setiap <i>frame</i>	Tiap 3 <i>frame</i>
Estimasi Posisi	Tidak ada	<i>Kalman Filter</i>

BAB III

Metodologi Penelitian

Metodologi Penelitian

- Lokasi: Sekretariat ERC Universitas Riau
- *Tools*: ROS, OpenCV, YOLOv8, NumPy, Arduino
- Tahapan:
 - Persiapan model YOLOv8
 - Implementasi *Baseline*
 - Implementasi *Optimized*
 - Pengujian 3 skenario

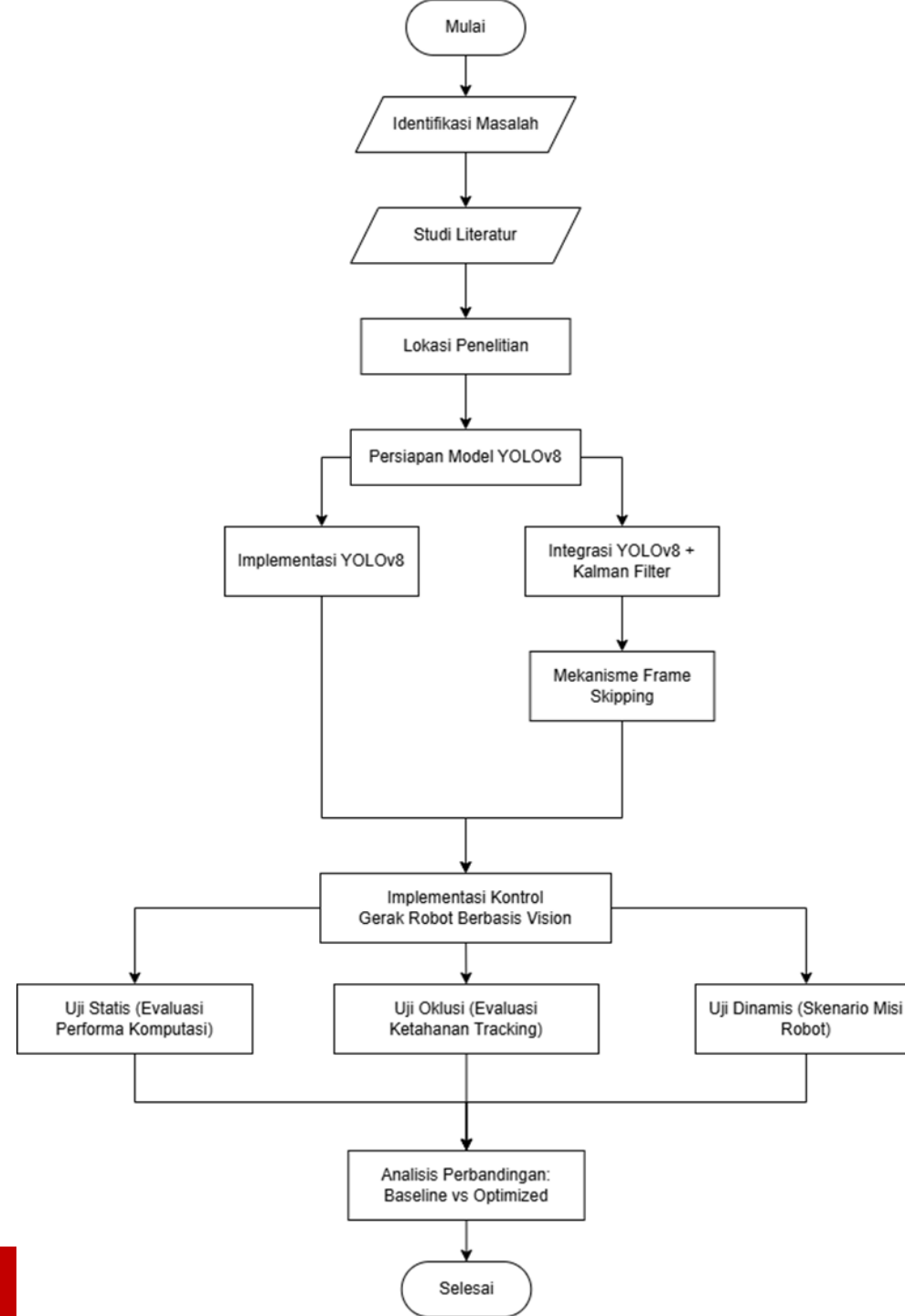


Diagram Sistem *Baseline*

- YOLOv8 (setiap *frame*) → Deteksi → Kontrol Gerak Robot

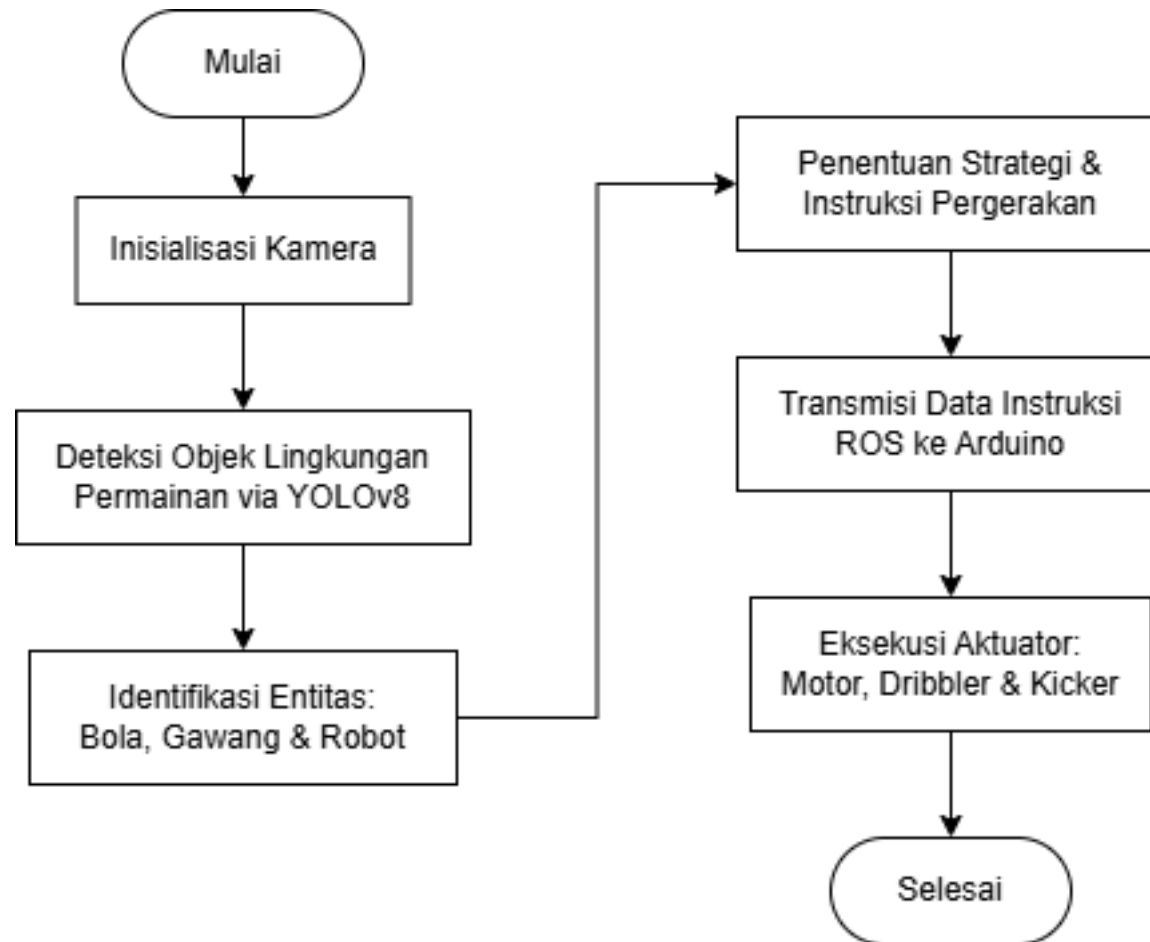
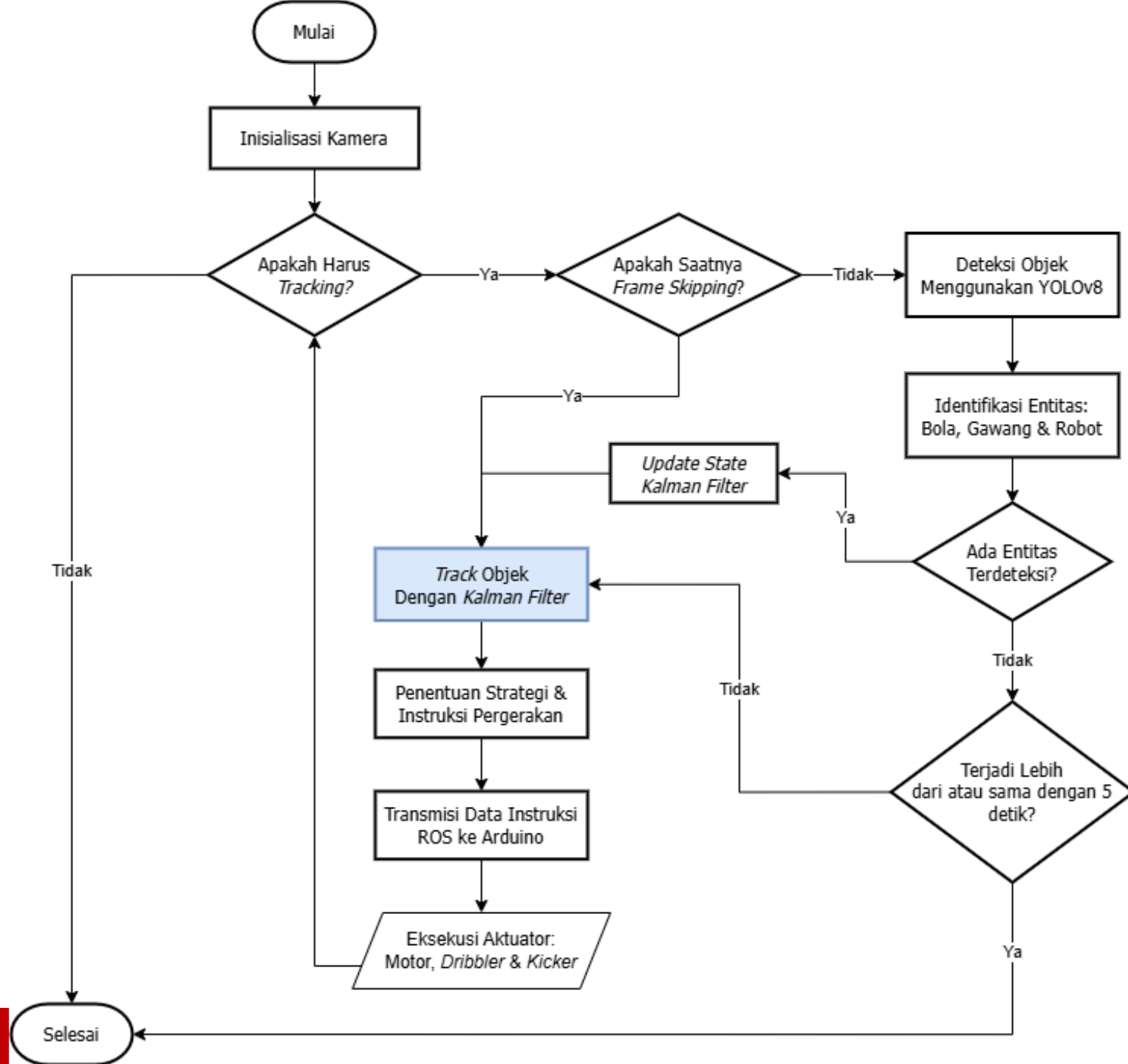


Diagram Sistem Optimized

- YOLOv8 (tiap 3 *frame* sekali) → *Kalman Filter Prediction (Tracking)* → Kontrol Gerak Robot



Skenario Pengujian (*Overview*)

- Penelitian ini melakukan pengujian pada 3 skenario berbeda untuk mengevaluasi performa sistem secara menyeluruh.
 - **Uji Statis** → Evaluasi performa komputasi (FPS)
 - **Uji Oklusi** → Evaluasi ketahanan *tracking*
 - **Uji Dinamis** → Evaluasi performa keseluruhan di lapangan
- Setiap skenario dilakukan berulang (repetisi) untuk mendapatkan data yang konsisten.

Uji Statis (Evaluasi Performa Komputasi)

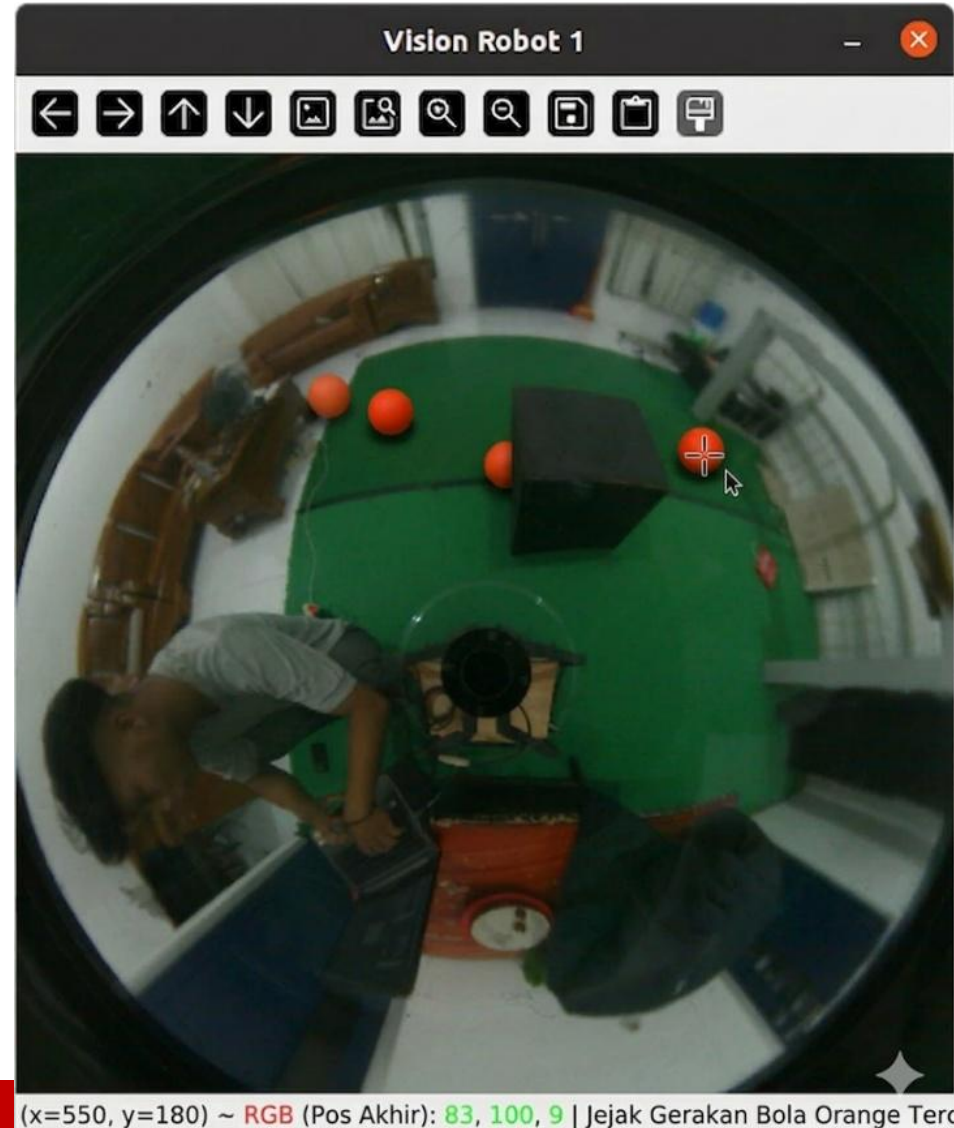
- **Tujuan:** Mengukur kecepatan pemrosesan sistem (*Frames Per Second / FPS*) pada *hardware* yang sama.
- **Cara Pengujian:**
 - Bola diletakkan statis di depan kamera
 - Sistem dijalankan selama beberapa menit
 - Diulang 10 kali untuk diambil rata-ratanya
 - Membandingkan ***Baseline*** (YOLOv8 setiap *frame*) vs ***Optimized*** (YOLOv8 + *Kalman Filter + Frame Skipping*)
- **Fokus Pengukuran:** Rata-rata FPS

Uji Oklusi (Evaluasi Ketahanan *Tracking*)

- **Tujuan:** Menguji ketahanan sistem saat bola tertutup (oklusi).
- **Cara Pengujian:**
 - Dilakukan dalam 3 tingkat oklusi: Ringan (30%), Sedang (60%), dan Sesaat
 - Bola sengaja ditutup sebagian atau sepenuhnya menggunakan penghalang
 - Diulang 10 kali per skenario
- **Parameter yang diukur:** *Tracking Loss Count* dan Total Durasi Kehilangan *Tracking*

Uji Oklusi (Evaluasi Ketahanan *Tracking*)

- Ilustrasi uji oklusi sesaat:



Uji Dinamis (Evaluasi Performa Keseluruhan)

- **Tujuan:** Menguji performa sistem dalam kondisi mendekati pertandingan nyata.
- **Cara Pengujian:**
 - Robot melakukan tugas lengkap: mencari bola → mendekati → mencetak gol
 - Dilakukan di lapangan dengan robot bergerak secara dinamis
 - Diulang 5 kali
- **Parameter yang diukur:** Waktu penyelesaian misi, stabilitas gerakan, dan keberhasilan mencetak gol (*Goal Rate*)

BAB IV

Hasil dan Pembahasan

Hasil Uji Statis



Hasil Uji Statis

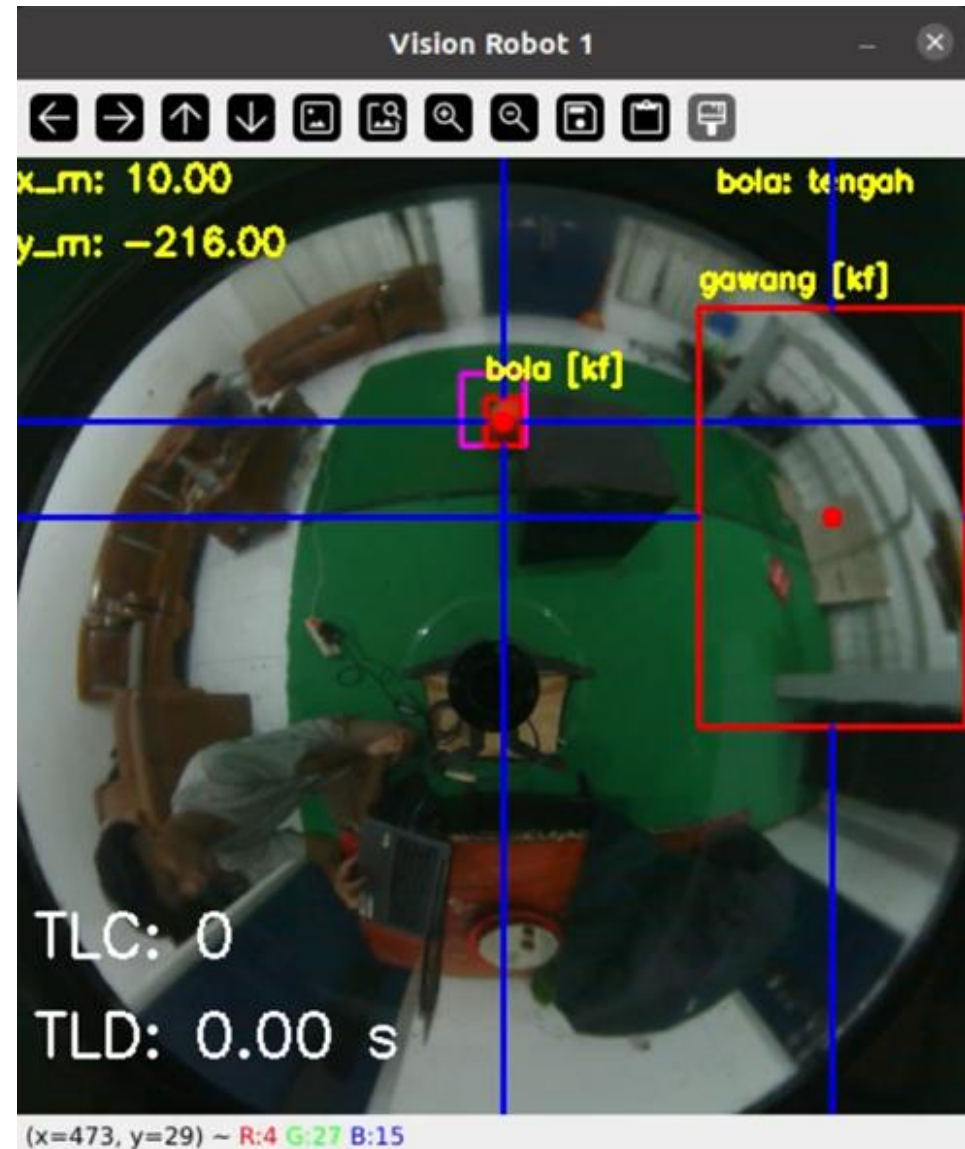
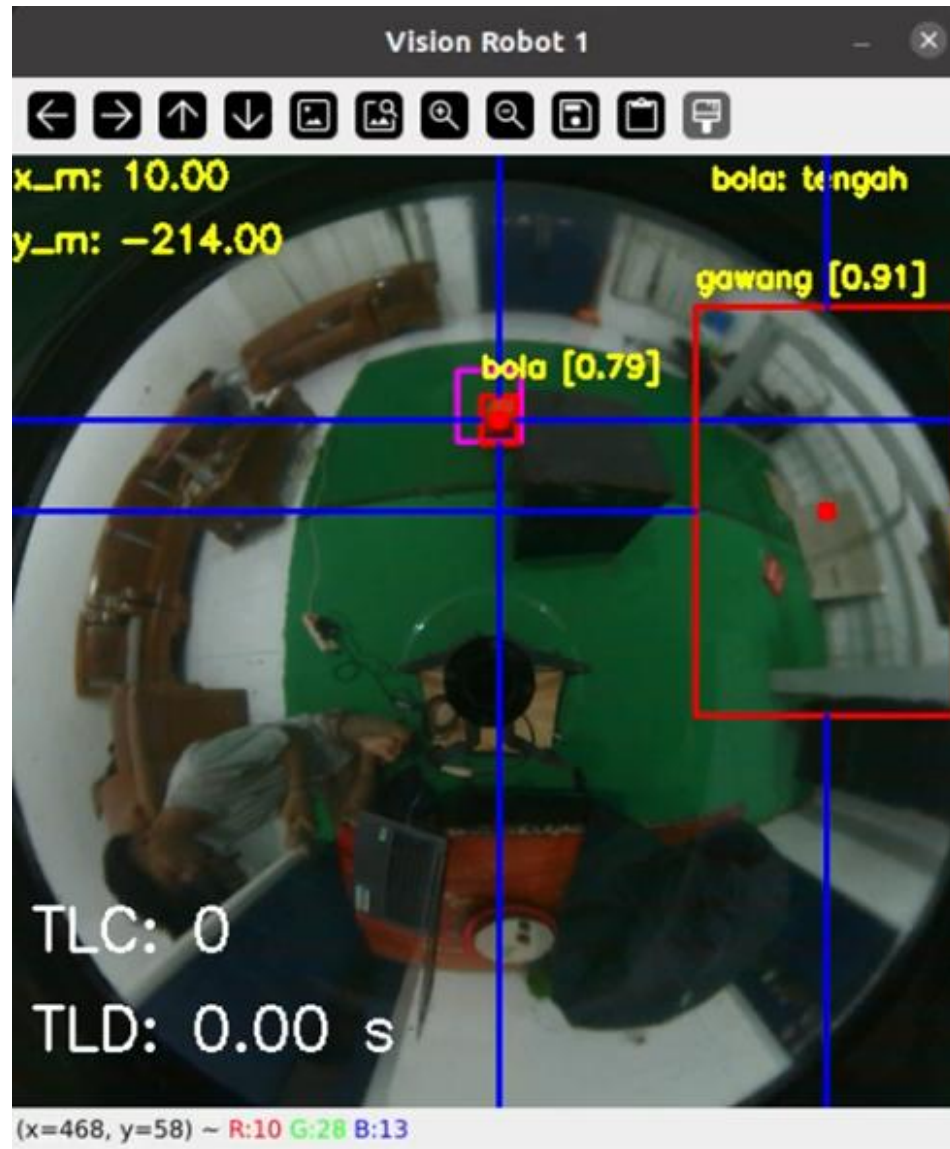
Repetisi	Sistem <i>Baseline</i>	Sistem <i>Optimized</i>
1	13.91	41.28
2	14.34	36.29
3	13.54	37.61
4	13.73	43.16
5	13.52	34.38

Repetisi	Sistem <i>Baseline</i>	Sistem <i>Optimized</i>
6	13.48	35.30
7	12.68	40.05
8	12.52	38.83
9	15.09	44.00
10	13.21	36.81
Rata-rata	13.60	38.77

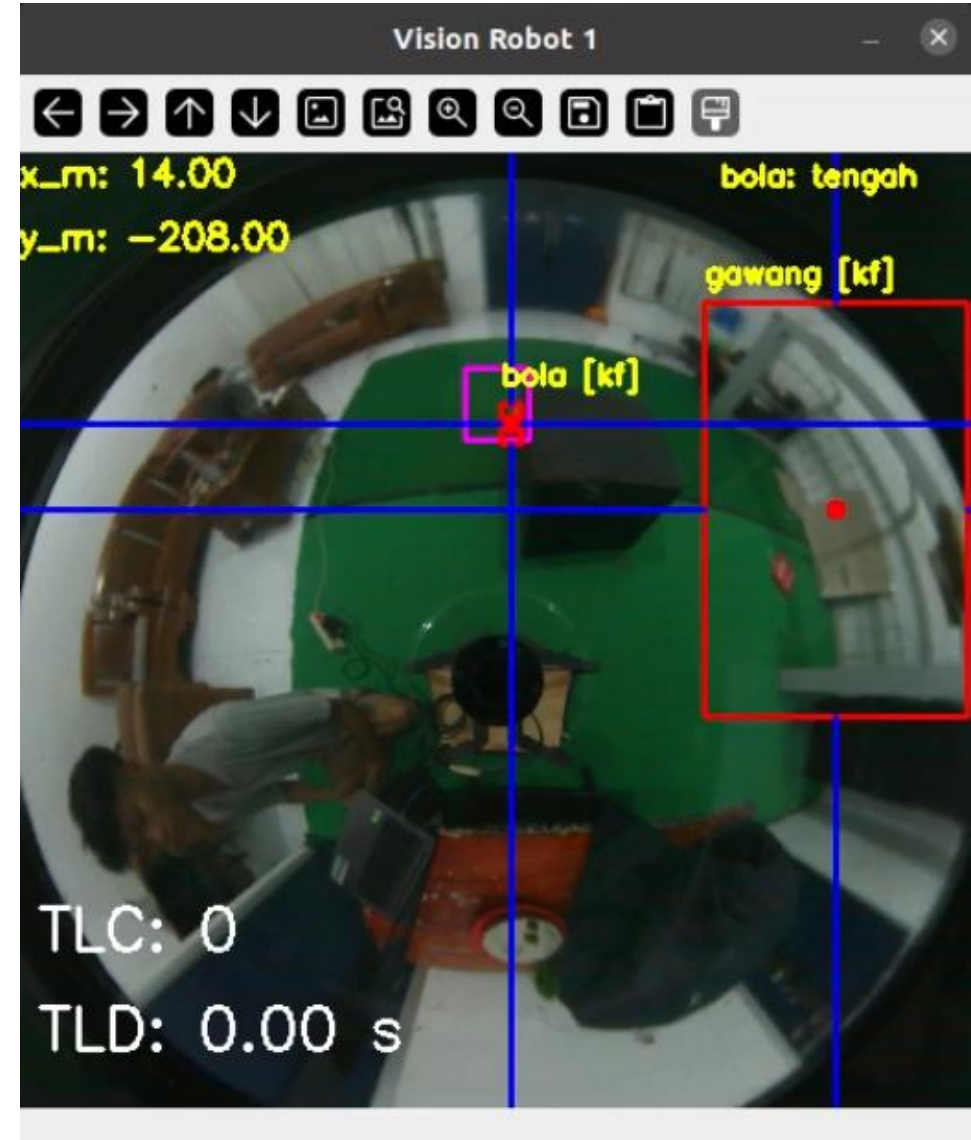
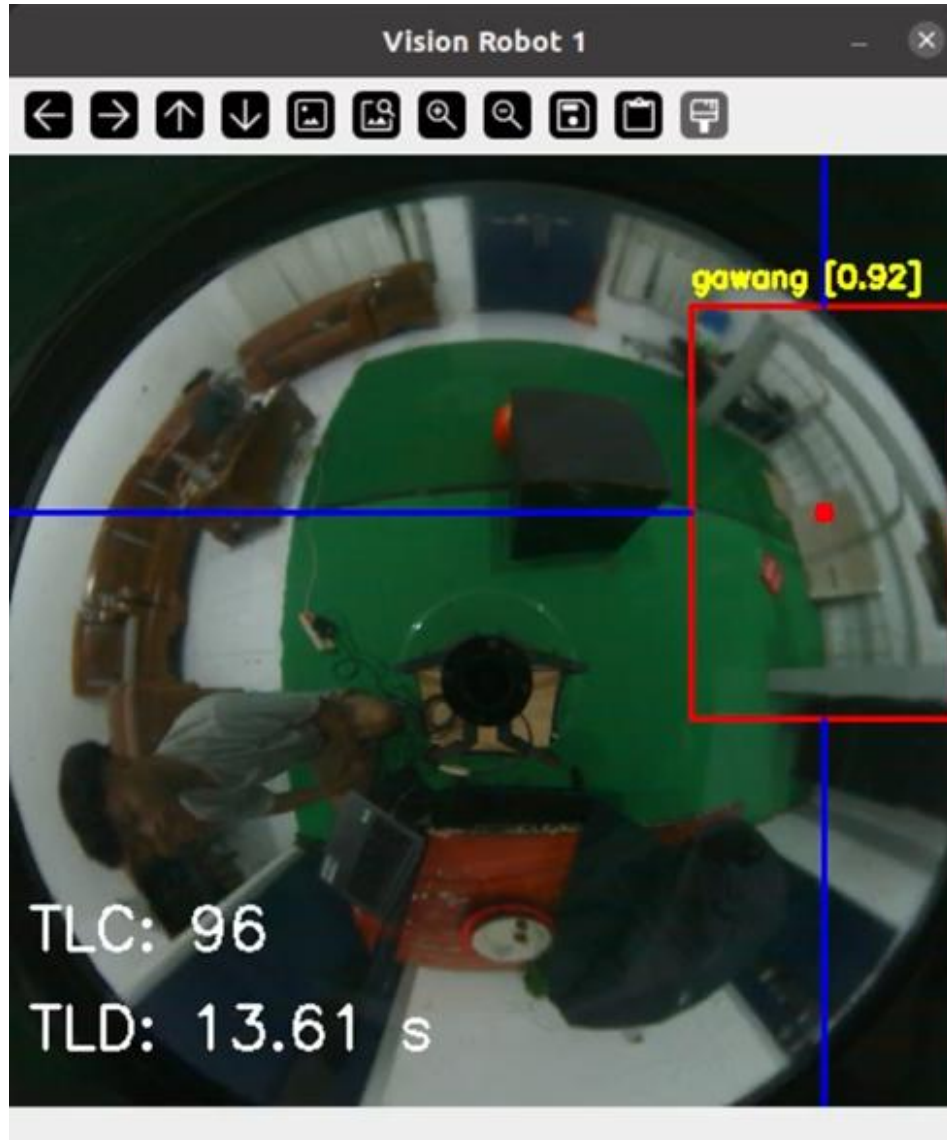
Hasil Uji Statis

- Uji statis menunjukkan peningkatan performa komputasi yang sangat signifikan pada sistem *Optimized*. Sistem *Baseline* hanya mampu mencapai rata-rata 13.60 FPS, sedangkan sistem *Optimized* berhasil mencapai rata-rata 38.77 FPS.
- Hal ini disebabkan karena YOLOv8 tidak perlu dijalankan pada setiap *frame*, melainkan hanya setiap 3 *frame*, sementara *Kalman Filter* mengisi prediksi posisi dengan komputasi yang sangat ringan.

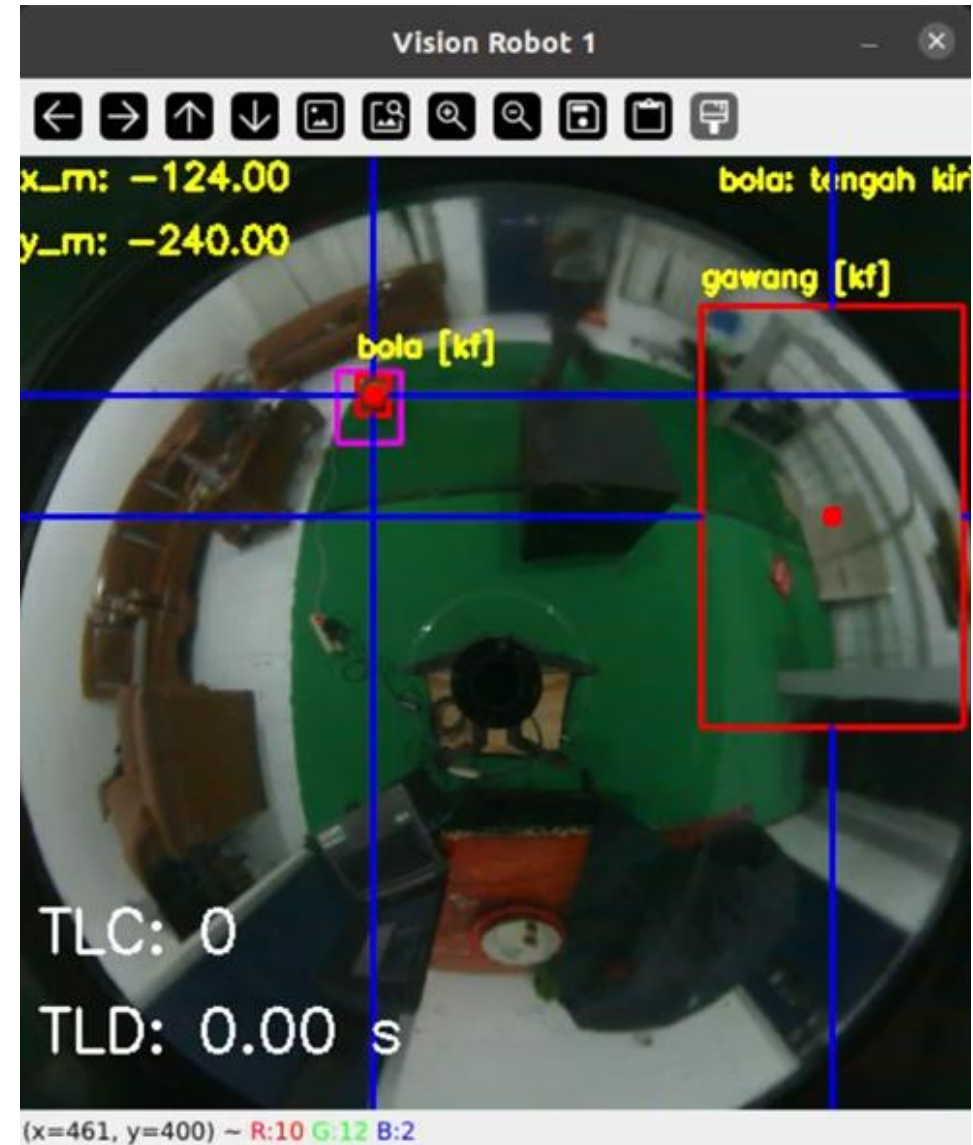
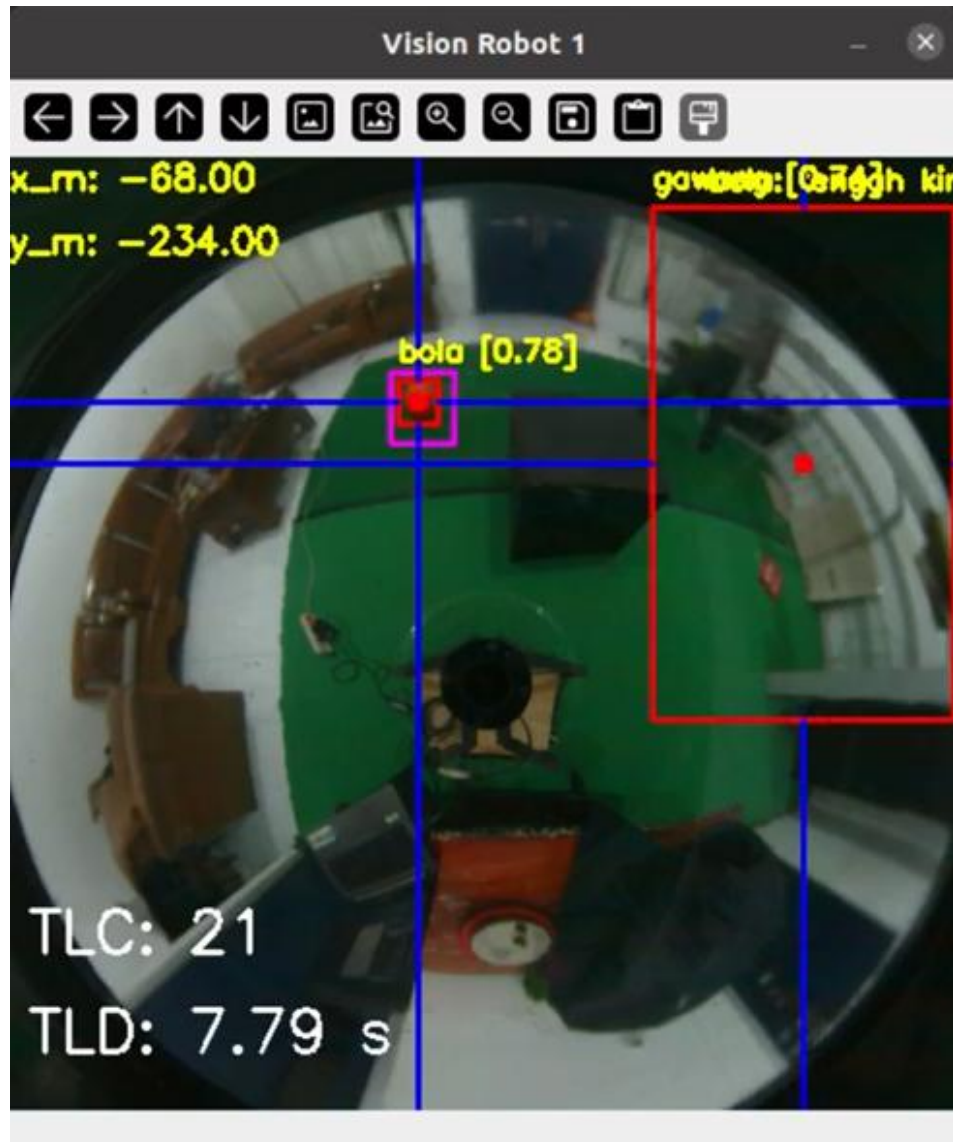
Hasil Uji Oklusi (Ringan)



Hasil Uji Oklusi (Sedang)



Hasil Uji Oklusi (Sesaat)



Hasil Uji Oklusi

- *Tracking Loss Count* (Jumlah Kejadian Kehilangan *Tracking*):

Skenario Uji	Sistem <i>Baseline</i>	Sistem <i>Optimized</i>
Oklusi ringan	0	0
Oklusi sedang	96	0
Oklusi sesaat	6	0

- *Tracking Loss Duration* (Durasi Kehilangan *Tracking*):

Skenario Uji	Sistem <i>Baseline</i>	Sistem <i>Optimized</i>
Oklusi ringan	0 detik	0 detik
Oklusi sedang	13.94 detik	0 detik
Oklusi sesaat	2.89 detik	0 detik

Hasil Uji Oklusi

- Pada uji oklusi, sistem *Optimized* menunjukkan ketahanan *tracking* yang jauh superior. Sistem ini berhasil mempertahankan pelacakan bola secara kontinu meskipun bola mengalami oklusi.
- Sistem *Baseline* sering kehilangan *tracking* ketika bola tertutup, sedangkan *Kalman Filter* mampu memprediksi posisi bola sehingga *tracking* tidak pernah hilang.

Hasil Uji Dinamis

```
timestamp,direction,topic,command
00:42:20.392,RECV,vision_topic,vision|ambilBola
00:42:20.398,SEND,cmd_status_topic_,received
00:42:20.398,SEND,vision_cmd_topic,hidupkanPenggiring
00:42:20.518,SEND,vision_cmd_topic,putarKiri
00:42:21.013,SEND,vision_cmd_topic,berhenti
00:42:21.166,SEND,vision_cmd_topic,putarKiri
00:42:21.239,SEND,vision_cmd_topic,berhenti
00:42:21.393,SEND,vision_cmd_topic,putarKiri
00:42:21.465,SEND,vision_cmd_topic,putarKiriPelan
00:42:23.068,SEND,vision_cmd_topic,maju
00:42:23.542,SEND,vision_cmd_topic,putarKananPelan
00:42:24.012,SEND,vision_cmd_topic,maju
00:42:24.490,SEND,vision_cmd_topic,putarKiriPelan
00:42:26.113,SEND,vision_cmd_topic,putarKiri
00:42:26.652,SEND,vision_cmd_topic,putarKiriPelan
00:42:26.725,SEND,vision_cmd_topic,majuPelan
00:42:26.882,SEND,vision_cmd_topic,putarKananPelan
00:42:28.575,SEND,vision_cmd_topic,maju
00:42:28.846,SEND,vision_cmd_topic,putarKiriPelan
00:42:30.101,SEND,vision_cmd_topic,maju
00:42:30.247,SEND,vision_cmd_topic,berhenti
00:42:30.320,SEND,vision_cmd_topic,maju
00:42:30.923,SEND,vision_cmd_topic,majuPelan
00:42:31.394,SEND,vision_cmd_topic,berhenti
00:42:31.394,SEND,cmd_status_topic_,[Vision] Selesai menjalankan perintah (vision_ambilBola)|1
00:42:31.416,RECV,vision_topic,vision|hadapGawang
00:42:31.418,SEND,cmd_status_topic_,received
00:42:31.418,SEND,vision_cmd_topic,hidupkanPenggiring
00:42:31.525,SEND,vision_cmd_topic,putarKiri
00:42:32.319,SEND,vision_cmd_topic,putarKiriPelan
00:42:32.792,SEND,vision_cmd_topic,berhenti
00:42:32.792,SEND,cmd_status_topic_,[Vision] Selesai menjalankan perintah (vision_hadapGawang)|1
00:42:36.641,RECV,vision_topic,vision|ambilBola
```

```
timestamp,direction,topic,command
00:47:00.819,RECV,vision_topic,vision|ambilBola
00:47:00.819,SEND,cmd_status_topic_,received
00:47:00.820,SEND,vision_cmd_topic,hidupkanPenggiring
00:47:00.841,SEND,vision_cmd_topic,berhenti
00:47:00.950,SEND,vision_cmd_topic,putarKiri
00:47:02.345,SEND,vision_cmd_topic,putarKiriPelan
00:47:03.349,SEND,vision_cmd_topic,maju
00:47:04.875,SEND,vision_cmd_topic,majuPelan
00:47:05.034,SEND,vision_cmd_topic,berhenti
00:47:05.035,SEND,cmd_status_topic_,[Vision] Selesai menjalankan perintah (vision_ambilBola)|1
00:47:05.053,RECV,vision_topic,vision|hadapGawang
00:47:05.056,SEND,cmd_status_topic_,received
00:47:05.058,SEND,vision_cmd_topic,hidupkanPenggiring
00:47:05.157,SEND,vision_cmd_topic,putarKanan
00:47:07.282,SEND,vision_cmd_topic,putarKananPelan
00:47:08.071,SEND,vision_cmd_topic,berhenti
00:47:08.073,SEND,cmd_status_topic_,[Vision] Selesai menjalankan perintah (vision_hadapGawang)|1
00:47:13.535,RECV,vision_topic,vision|ambilBola
00:47:13.535,SEND,cmd_status_topic_,received
00:47:13.535,SEND,vision_cmd_topic,hidupkanPenggiring
00:47:13.632,SEND,vision_cmd_topic,putarKananPelan
00:47:13.771,SEND,vision_cmd_topic,putarKanan
00:47:13.876,SEND,vision_cmd_topic,putarKananPelan
00:47:15.379,SEND,vision_cmd_topic,maju
00:47:16.901,SEND,vision_cmd_topic,majuPelan
00:47:17.533,SEND,vision_cmd_topic,berhenti
00:47:17.534,SEND,cmd_status_topic_,[Vision] Selesai menjalankan perintah (vision_ambilBola)|1
00:47:17.581,RECV,vision_topic,vision|hadapGawang
00:47:17.581,SEND,cmd_status_topic_,received
00:47:17.581,SEND,vision_cmd_topic,hidupkanPenggiring
00:47:17.595,SEND,vision_cmd_topic,putarKiri
00:47:18.375,SEND,vision_cmd_topic,putarKiriPelan
00:47:18.888,SEND,cmd_status_topic_,[Vision] Selesai menjalankan perintah (vision_hadapGawang)|1
```

Hasil Uji Dinamis

- Waktu Penyelesaian Misi:

Repetisi	Sistem <i>Baseline</i>	Sistem <i>Optimized</i>
1	12.400 detik	7.254 detik
2	19.277 detik	5.350 detik
3	12.986 detik	5.987 detik
4	19.139 detik	5.615 detik
5	12.057 detik	4.638 detik
Rata-rata	15.171 detik	5.768 detik

Hasil Uji Dinamis

- Stabilitas Gerakan dan Koreksi Arah (*Jitter*):

Jenis Perintah	Total Kemunculan Sistem <i>Baseline</i>	Total Kemunculan Sistem <i>Optimized</i>	Reduksi
putarKiri / putarKiriPelan	44 kali	11 kali	-33 kali
putarKanan / putarKananPelan	30 kali	12 kali	-18 kali
maju / majuPelan	48 kali	10 kali	-38 kali
Total Command Switching	122 kali	33 kali	-73%

Hasil Uji Dinamis

- Keberhasilan Eksekusi (*Goal Rate*):

Repetisi	Sistem <i>Baseline</i>	Sistem <i>Optimized</i>
1	Gol	Gol
2	Gol	Gol
3	Gol	Gol
4	Gol	Gol
5	Gol	Gol
<i>Goal Rate</i>	100%	100%

Hasil Uji Dinamis

- Dalam uji dinamis, sistem *Optimized* menunjukkan performa yang jauh lebih baik. Robot berhasil menyelesaikan misi mencetak gol dengan waktu yang jauh lebih singkat dan gerakan yang jauh lebih stabil.
- Integrasi *Kalman Filter* berhasil mengurangi *jitter* pada pergerakan robot, sehingga robot dapat mengambil keputusan arah lebih cepat dan akurat.

Ringkasan Hasil Keseluruhan

- **Perbandingan Keseluruhan**
 - FPS: 13.60 → 38.77 (+185%)
 - *Tracking Loss Count*: 102 → 0
 - **Durasi Tracking Loss**: >15 detik → 0 detik
 - **Waktu Mencetak Gol**: 15.171 detik → 5.768 detik (2.63x lebih cepat)
 - **Reduksi jitter**: 73% (*command switching* dari 122 kali menjadi 33 kali)
- **Kesimpulan Singkat**: Sistem *Optimized* menghasilkan pergerakan dan sistem *tracking* robot yang lebih cepat, stabil, dan tangguh terhadap oklusi.

BAB V

Penutup

Kesimpulan

- Penelitian ini berhasil mengoptimasi gerakan robot KRSBI Beroda melalui integrasi **YOLOv8 + Kalman Filter + Frame Skipping**.
- **Pencapaian Utama:**
 - Peningkatan FPS sebesar **185%** (13.60 → 38.77 FPS)
 - *Tracking Loss Count 0* di semua skenario oklusi
 - Waktu mencetak gol **2.63x lebih cepat** (15.171 → 5.768 detik)
 - **Reduksi jitter hingga 73%** (*command switching* dari 122 kali menjadi 33 kali)
 - *Goal Rate* tetap **100%**
- **Kesimpulan Inti:**
 - Integrasi *Kalman Filter* tidak hanya meningkatkan kecepatan dan ketahanan *tracking*, tetapi juga **sangat efektif mereduksi jitter**, sehingga menghasilkan gerakan robot yang jauh lebih stabil dan efisien pada *hardware* terbatas.

Saran

- Implementasi pada *hardware* yang lebih kuat (misal. Nano Jetson)
- Tambahkan *multi-object tracking* (bola + robot lawan)
- Pengujian di kondisi pencahayaan yang lebih variatif
- Coba algoritma *Filter* selain *Kalman Filter* (misal. *Extended Kalman Filter*)

TERIMAKASIH